



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Cultura i Esport



UNIÓ EUROPEA

Fons Social Europeu
L'FSE inverteix en el teu futur

IES ENRIC SOLER I GODES

Projecte d'Administració de Sistemes Informàtics en Xarxa

Curs: 2025-2026

SISTEMAS DE MONITORIZACIÓN

Autor: Kevin Murciano Gadea | Sergio Ferrá Boix

NIA: 10788620 | 10663813

Data: 23/03/26

Tutoria de projecte: Isaac García Simarro

Índice

1. Introducción.....	3
2. Objetivos.....	4
3. Análisis de requerimientos.....	4
3.1 Necesidades, requerimientos del cliente y análisis de la situación actual.....	4
3.2 Estudio de alternativas de solución.....	5
3.3 Elección, valoración económica y diseño de las posibles soluciones.....	6
4. Implementación.....	7
4.1 Diagramas y Esquemas:.....	7
4.2 Herramientas de software.....	8
4.3 Herramientas de hardware.....	9
4.4 Lenguajes.....	9
4.5 Codificación:.....	10
4.6 Administración.....	13
4.7. Aspectos de seguridad.....	14
5. Fases en la realització del projecte.....	14
Fase 1: Adecuación del entorno en Proxmox.....	14
Fase 2: Vigilancia de disponibilidad (Uptime Kuma).....	15
Fase 3: Monitorización avanzada con Pandora FMS.....	16
Fase 4: Integración de alertas con Gotify.....	17
Fase 5: Gestión centralizada de credenciales con KeePass.....	19
Fase 6: Documentación y repositorio en GitHub.....	20
6. Ampliacions.....	21
7. Conclusions.....	22
8. Bibliografia.....	23
9. Índice de figuras y capturas.....	24

1. Introducción

Hoy en día, las empresas dependen totalmente de sus ordenadores y sistemas para trabajar. Si un servidor se rompe o internet deja de funcionar, la actividad se detiene y se pierde dinero. Por eso, es fundamental tener "ojos" que vigilen todo lo que pasa en la red las 24 horas del día. A esto lo llamamos Monitorización.

Este proyecto consiste en la creación de un centro de control desde cero. Para ello, no nos hemos limitado a instalar un programa, sino que hemos preparado un servidor físico real y lo hemos dividido en varios compartimentos virtuales gracias a una herramienta llamada Proxmox.

Dentro de estos compartimentos, hemos instalado distintos programas profesionales (como Uptime Kuma o Pandora FMS) que se encargan de avisarnos si algo falla. El objetivo final es pasar de un modelo donde "arreglamos las cosas cuando se rompen" a uno donde "sabemos que algo va a fallar antes de que ocurra", garantizando que todo funcione de forma fluida y segura.

Para llevar a cabo este proyecto hemos necesitado conocimientos adquiridos en varios módulos del ciclo. Del módulo de Sistemas Operativos utilizamos los conceptos de virtualización: durante el curso instalamos y configuramos servidores, y aunque el entorno Proxmox que usamos en el proyecto fue preparado previamente por el profesor, la experiencia con esa herramienta nos permitió movernos con soltura dentro de él. Del módulo de Redes aplicamos los fundamentos de comunicación entre sistemas y los conocimientos sobre Docker, que nos sirvieron de base para entender cómo desplegar servicios en contenedores dentro de la infraestructura. Del módulo de Seguridad aplicamos los conceptos de alta disponibilidad y redundancia de servicios. El uso de Keepalived para la gestión de la IP virtual mediante el protocolo VRRP nos permitió eliminar el punto único de fallo del portal web, asegurando que si el servidor principal cae, el secundario toma el relevo de forma automática y transparente. Esto conecta directamente con los principios de disponibilidad trabajados en el módulo, donde la continuidad del servicio es uno de los pilares fundamentales de cualquier infraestructura segura.

2. Objetivos

El objetivo principal de este trabajo es desplegar un sistema de monitorización completo que vigile automáticamente el estado de los equipos y servicios informáticos, garantizando que estén siempre disponibles y que cualquier problema sea detectado y notificado antes de que afecte al funcionamiento.

Los objetivos específicos son los siguientes:

- Montar y configurar un servidor físico real sobre el que desplegar toda la infraestructura de virtualización, utilizando Proxmox VE como plataforma base.
- Instalar y configurar herramientas de monitorización profesionales (Uptime Kuma y Pandora FMS) capaces de supervisar el estado de servicios y equipos en tiempo real.
- Implementar un sistema de alertas que notifique automáticamente al equipo técnico cuando se detecte un fallo o una anomalía, mediante Gotify.
- Garantizar la seguridad del entorno configurando accesos restringidos.
- Validar el funcionamiento del sistema simulando fallos reales y comprobando que la detección y notificación se producen correctamente.
- Documentar todo el proceso de instalación y configuración para que cualquier persona pueda mantener o reproducir el sistema en el futuro.

3. Análisis de requerimientos

3.1 Necesidades, requerimientos del cliente y análisis de la situación actual

El punto de partida es una infraestructura sin ningún sistema de monitorización centralizado. Los equipos son gestionados de forma reactiva: las incidencias se detectan únicamente cuando los usuarios las reportan, lo que conlleva tiempos de resolución elevados y pérdida de productividad.

Los requerimientos principales identificados son: (a) supervisión continua del estado de las máquinas virtuales y los servicios web, (b) notificaciones inmediatas ante caídas o degradaciones de rendimiento, y (c) registro histórico de métricas para análisis posterior.

3.2 Estudio de alternativas de solución

Para el diseño de la infraestructura, se han evaluado diferentes grupos de soluciones técnicas para cada componente crítico de la arquitectura, analizando su viabilidad, costes y facilidad de implementación:

Virtualización: Las alternativas consideradas fueron VMware ESXi (solución propietaria, licencia de coste elevado), Hyper-V (integrado en Windows Server, requiere licencia) y Proxmox VE (código abierto, basado en Debian Linux y KVM).

- Se optó por **Proxmox VE** por su coste cero, su comunidad activa y su interfaz web de administración completa.

Monitorización de disponibilidad: Se compararon Nagios (potente pero complejo de configurar), Zabbix (solución completa pero con alta curva de aprendizaje) y Uptime Kuma (interfaz moderna, fácil despliegue en Docker).

- Se eligió **Uptime Kuma** por su sencillez de configuración y su integración nativa con múltiples canales de alerta.

Monitorización avanzada de rendimiento: Se valoraron Zabbix, Checkmk y Pandora FMS.

- Se seleccionó **Pandora FMS** por su versión comunitaria gratuita, su soporte de agentes SNMP y la capacidad de generar informes detallados.

Sistema de notificaciones: Las alternativas analizadas fueron Telegram Bot (dependencia de terceros), ntfy (autoalojado, más básico) y Gotify (autoalojado, aplicación Android oficial, API REST sencilla).

- Se eligió **Gotify yntfy** por su total independencia de servicios externos y su integración con Uptime Kuma.

3.3 Elección, valoración económica y diseño de las posibles soluciones

El coste de implantación de este proyecto es prácticamente nulo gracias a dos factores clave: la donación del hardware y el uso exclusivo de software de código abierto.

Hardware El servidor físico sobre el que se ejecuta Proxmox VE ha sido donado por una de las empresas en las que los autores realizan el módulo de Formación en Centros de Trabajo (FCT). Esta cesión elimina por completo el principal coste de cualquier infraestructura de virtualización.

No obstante, para una implementación desde cero, un servidor con las características necesarias para este proyecto (procesador de gama media-profesional, 32GB de RAM y almacenamiento redundante SSD) tendría un **coste de mercado estimado de entre 800 € y 1.200 €**.

Software Todas las herramientas utilizadas en el proyecto son gratuitas y de código abierto:

- **Proxmox VE:** hipervisor de tipo 1 publicado bajo licencia AGPL. Su descarga e instalación son completamente gratuitas.
- **Uptime Kuma:** publicado bajo licencia MIT. No existe versión de pago; toda la funcionalidad está disponible sin coste.
- **Gotify:** publicado bajo licencia MIT. Servidor de notificaciones autoalojado sin ningún modelo de suscripción.
- **Pandora FMS Community:** se utiliza la edición Community (licencia GPL). Cabe destacar que, aunque existe una versión Enterprise de pago con soporte oficial y módulos avanzados, la versión gratuita cubre todas las necesidades de este análisis.

Componente	Solución elegida	Coste Real	Coste Estimado (Compra)
Servidor físico	Prestado por el profesorado	0€.	~1000€.
Hipervisor	Proxmox VE (AGPL)	0€.	0€.
Monitorización disponibilidad	Uptime Kuma (MIT)	0€.	0€.
Notificaciones push	Gotify/ntfy/Telegram (MIT)	0€.	0€.
Monitorización rendimiento	Pandora FMS Community	0€.	0€.
TOTAL		0€.	~1000€.

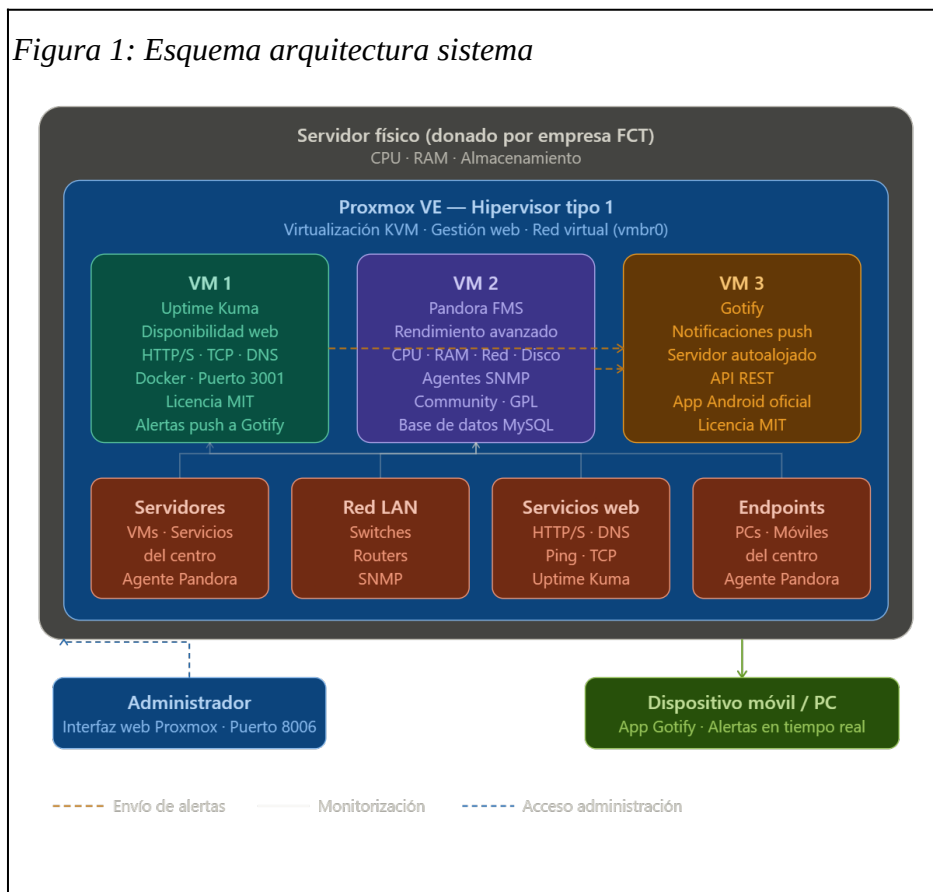
4. Implementación

En este apartado se detalla el despliegue técnico del sistema. La solución propuesta no es una elección arbitraria, sino la respuesta directa a las necesidades identificadas: se ha optado por un **entorno virtualizado con Proxmox VE** para garantizar flexibilidad y aislamiento, el uso exclusivo de **herramientas de código abierto** para eliminar costes de licencias, y una arquitectura de **servicios independientes** (Uptime Kuma, Pandora FMS y Gotify) para cubrir desde la disponibilidad inmediata hasta el análisis de rendimiento avanzado y la gestión de alertas sin dependencias externas.

4.1 Diagramas y Esquemas:

A continuación se presenta el esquema general de la arquitectura del sistema de monitorización implementado. El proyecto se estructura en tres capas principales: el hardware físico, la capa de virtualización gestionada por Proxmox VE, y los servicios de monitorización desplegados en máquinas virtuales independientes

Figura 1: Esquema arquitectura sistema



La figura 1 muestra cómo el servidor físico aloja el hipervisor Proxmox VE, que a su vez gestiona tres máquinas virtuales: una para Uptime Kuma (monitorización de disponibilidad), otra para Pandora FMS (monitorización avanzada de rendimiento) y una tercera para Gotify (servidor de notificaciones push autoalojado). Uptime Kuma y Pandora FMS envían alertas a Gotify, que distribuye las notificaciones push a los dispositivos móviles y ordenadores de los administradores.

Component	Funció principal
Servidor físico (hardware) curl -SSL https://pfms.me/de	Base de cómputo: CPU, RAM y almacenamiento para todas las VMs.
Proxmox VE (hipervisor)	Virtualización de tipo 1 basada en KVM. Gestión de VMs desde interfaz web.
VM 1 — Uptime Kuma	Monitorización de disponibilidad de servicios web y endpoints (HTTP/S, TCP, DNS).
VM 2 — Pandora FMS Community	Monitorización avanzada de rendimiento: CPU, RAM, red, disco. Agentes SNMP.
VM 3 — Gotify	Servidor de notificaciones push autoalojado. App Android oficial. API REST.

4.2 Herramientas de software

El proyecto ha utilizado exclusivamente herramientas de código abierto. No se ha adquirido ninguna licencia de software de pago.

- **Proxmox VE 8.x:** Hipervisor de tipo 1 basado en Debian 12 y KVM. Permite la gestión de VMs y contenedores LXC mediante interfaz web (Licencia AGPL).
- **Ubuntu Server 24.04:** Sistema operativo base de las máquinas virtuales donde se instalan los servicios de Gotify y Uptime Kuma.
- **Almalinux 9:** Sistema operativo recomendado para desplegar Pandora FMS.
- **Uptime Kuma 1.x:** Aplicación de monitorización de tiempo de actividad desplegada en Docker. Soporta HTTP/S, TCP, ping, DNS y SNMP (Licencia MIT).
- **Docker Engine:** Plataforma de contenedores utilizada para el despliegue de Uptime Kuma.
- **Pandora FMS Community Edition:** Plataforma de monitorización empresarial en versión comunitaria gratuita. Supervisa CPU, RAM, disco y red mediante agentes (Licencia GPL).
- **MySQL / MariaDB:** Base de datos utilizada por Pandora FMS para almacenar métricas y configuración.
- **Gotify:** Servidor de notificaciones push autoalojado con API REST y aplicación oficial para Android (Licencia MIT).
- **Otras herramientas:** Git para control de versiones, cURL/wget para pruebas de conectividad y nano/vim como editores de texto.
- **Revisión del servidor físico:** Verificación de los componentes (CPU, RAM y Almacenamiento) para asegurar que soportarían la carga de tres máquinas virtuales simultáneas.

4.3 Herramientas de hardware

El servidor físico es el elemento central de la infraestructura. Fue donado por una de las empresas donde los autores realizan el módulo de Formación en Centros de Trabajo (FCT).

Componente de hardware	Detalle / Configuración
Procesador (CPU)	Arquitectura x86-64 con soporte de virtualización (Intel VT-x / AMD-V).
Memoria RAM	Capacidad suficiente para alojar Proxmox VE y tres máquinas virtuales simultáneas.
Almacenamiento	Disco duro o SSD local. Proxmox gestiona el pool (LVM o ZFS).
Interfaz de red	Tarjeta Ethernet integrada. Proxmox crea un bridge virtual (vbr0) para la LAN.
Conexión a la red	El servidor se conecta a la LAN del centro, permitiendo acceso remoto a la interfaz web.

Configuración del hardware: Se verificaron físicamente los componentes, se activó la virtualización en la BIOS/UEFI y se instaló Proxmox VE mediante USB, configurando una IP estática y el bridge de red virtual.

4.4 Lenguajes

El proyecto se centra en la administración de sistemas, pero se han empleado diversos lenguajes y formatos para la automatización y configuración:

- **Bash / Shell scripting:** Para automatizar la instalación de paquetes, arranque de Docker y copias de seguridad.
- **YAML:** Formato utilizado para la exportación y restauración de configuraciones en Uptime Kuma.
- **JSON:** Estructura de los mensajes para la API REST de Gotify.
- **INI / ficheros de configuración:** Utilizados por Pandora FMS para definir agentes y parámetros de conexión.
- **HTML y CSS:** Utilizados internamente por las consolas web de las herramientas.

4.5 Codificación:

A continuación se muestran los scripts más relevantes. El resto de ficheros se encuentran en el soporte físico adjunto.

Script de instalación de Uptime Kuma (install_uptime_kuma.sh): Automatiza la actualización del sistema, la instalación de Docker y la ejecución del contenedor de Uptime Kuma con persistencia de datos.

```
#!/bin/bash
# Instalación de Docker + Uptime Kuma
# Compatible con Ubuntu Server 24.04
# Ejecutar con privilegios de superusuario (root)

set -e # Finaliza el script si algún comando devuelve un error

# 1. Instalar dependencias del sistema
apt-get update -y
apt-get install -y ca-certificates curl gnupg

# 2. Configurar el repositorio oficial de Docker
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
        https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo \
"$VERSION_CODENAME") stable" | \
    tee /etc/apt/sources.list.d/docker.list > /dev/null

# 3. Instalar Docker Engine y plugins
apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 4. Habilitar y arrancar el servicio de Docker
```

```
systemctl enable --now docker
```

```
# 5. Desplegar contenedor de Uptime Kuma
```

```
# Se utiliza un volumen persistente para no perder la configuración al reiniciar
```

```
docker run -d \
```

```
--restart=always \
```

```
-p 3001:3001 \
```

```
-v uptime-kuma:/app/data \
```

```
--name uptime-kuma \
```

```
louislam/uptime-kuma:1
```

```
echo ""
```

```
echo "Instalación completada con éxito."
```

```
echo "Accede a la interfaz web en: http://$(hostname -I | awk '{print $1}'):3001"
```

Script instalación de Pandora FMS (pandorafms_install.sh): Despliega pandora FMS con la versión gratuita.

```
#!/bin/bash
```

```
# Instalación de Pandora FMS - Entorno Monolítico (EL9)
```

```
# Doc oficial: https://pandorafms.com/manual/current/es/documentation/pandorafms/installation/01\_installing
```

```
# Ejecutar como root en EL9 (RHEL 9 / Rocky Linux 9 / AlmaLinux 9)
```

```
curl -sS https://pfms.me/deployenterprise | \
```

```
TZ='Europe/Madrid' \
```

```
DBHOST='127.0.0.1' \
```

```
DBNAME='pandora' \
```

```
DBUSER='pandora' \
```

```
DBPASS='tucontraseñasegura99abc!' \
```

```
DBPORT='3306' \
```

```
DBROOTPASS='tucontraseñasegura99abc!' \
```

```
MYVER=80 \
```

```
PHPVER=8 \
```

```
SKIP_PRECHECK=0 \
```

```
SKIP_DATABASE_INSTALL=0 \
```

```
SKIP_KERNEL_OPTIMIZATIONS=0 \
```

```
PANDORA_AGENT_PACKAGE="https://firefly.pandorafms.com/pandorafms/latest/
pandorafms_one_agent_linux_bin-latest.el9.x86_64.rpm" \
PANDORA_BETA=0 \
PANDORA_LTS=1 \
PANDORA_ENT_FREE=1 \
bash
```

Script instalación de gotify:

```
#!/bin/bash
# Instalación de Docker + Gotify
# Compatible con Ubuntu Server 24.04
# Ejecutar con privilegios de superusuario (root)

set -e # Finaliza el script si algún comando devuelve un error

# 1. Instalar dependencias del sistema
apt-get update -y
apt-get install -y ca-certificates curl gnupg

# 2. Configurar el repositorio oficial de Docker
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
        https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
    tee /etc/apt/sources.list.d/docker.list > /dev/null

# 3. Instalar Docker Engine y plugins
apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-
plugin
```

```
# 4. Habilitar y arrancar el servicio de Docker
```

```
systemctl enable --now docker
```

```
# 5. Desplegar contenedor de Gotify
```

```
# Se utiliza un volumen persistente para guardar los mensajes y la configuración
```

```
docker run -d \
```

```
--restart=always \
```

```
-p 8080:80 \
```

```
-v gotify-data:/app/data \
```

```
--name gotify \
```

```
gotify/server
```

```
echo ""
```

```
echo "Instalación completada con éxito."
```

```
echo "Accede a la interfaz web en: http://$(hostname -I | awk '{print $1}'):8080"
```

```
echo "Credenciales por defecto: usuario 'admin', contraseña 'admin'"
```

4.6 Administración

El sistema se gestiona a través de tres interfaces web independientes:

- **Proxmox VE:** Puerto 8006 (Gestión global).
- **Uptime Kuma:** Puerto 3001 (Panel de monitores).
- **Pandora FMS:** /pandora_console (Consola completa).
- **Gotify:** Puerto 8080 (Gestión de tokens).
- **NTFY:** Puerto 8081

Se han definido perfiles de usuario como **root/admin** para acceso total y perfiles de **monitorización/operador** para tareas de solo lectura, garantizando la seguridad y trazabilidad.

4.7. Aspectos de seguridad

Para garantizar la integridad del sistema y la protección de los datos de monitorización recopilados, se han implementado una serie de medidas y protocolos de seguridad que blindan la infraestructura frente a posibles amenazas externas e internas:

- **Prevención de accesos:** Uso de contraseñas robustas (mínimo 12 caracteres) y restricción de acceso a las consolas web únicamente desde la LAN del centro; ningún puerto es accesible desde Internet.
- **Aislamiento y Actualización:** Cada servicio corre en una VM independiente para contener posibles fallos. Se mantienen actualizados tanto el SO como los contenedores Docker.
- **Protección de aplicaciones:** Las herramientas incluyen protección nativa contra fuerza bruta y vulnerabilidades web como XSS o inyección SQL.

5. Fases en la realizació del projecte

El proyecto se ha desarrollado siguiendo un enfoque progresivo, desde la adecuación de la infraestructura base hasta la validación final del sistema de alertas.

Fase 1: Adecuación del entorno en Proxmox

En un principio, nuestra intención era montar y configurar el servidor físico desde cero. Sin embargo, tras hablarlo con los profesores, nos aconsejaron utilizar la infraestructura de **Proxmox** que ya estaba preparada en el aula. Esto fue una buena decisión, ya que nos permitió ganar tiempo y centrar nuestros esfuerzos en el despliegue y la configuración de las herramientas de monitorización.

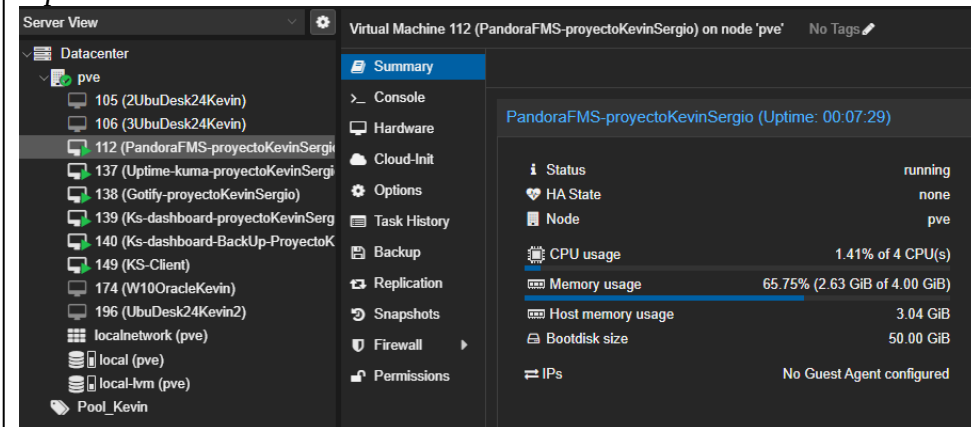
- **Creación de las VMs:** Dentro del entorno Proxmox de clase, creamos tres máquinas virtuales independientes. De esta manera, cada servicio (Uptime Kuma, Pandora y Gotify) tiene su propio espacio y no hay conflictos entre ellos.
- **Configuración de Red e IPs:** Lo más importante aquí fue asignar **IPs fijas** a cada máquina. En un sistema de monitorización no podemos usar IPs automáticas (DHCP), porque si la IP cambia, los sensores dejarían de encontrar los servidores y el sistema fallaría.

Nuestra red de trabajo quedó configurada así:

- **IP Proxmox (Servidor de clase):** 10.2.1.253
- **VM1 - Uptime Kuma:** 10.2.1.168
- **VM2 - Pandora FMS:** 10.2.1.167
- **VM3 – Alertas (Gotify):** 10.2.1.169
- **VM4 – KS-Client (Cliente Ubuntu):** 10.2.1.163

- **VM5- KS-Dashboard (VM): 10.2.1.166 y (Web): 10.2.1.165**
- **VM6 – KS-Dashboard-Backup (VM): 10.2.1.164 y (Web): 10.2.1.165**

Captura 1: VM's Proxmox



Fase 2: Vigilancia de disponibilidad (Uptime Kuma)

Con las máquinas ya listas, empezamos instalando **Uptime Kuma** en la primera VM. Este programa es nuestra primera línea de defensa: nos dice de forma visual y rápida si un servicio está "vivo" o se ha caído.

- **Despliegue con Docker:** Para que la instalación fuera limpia y profesional, usamos Docker. Es mucho más cómodo porque el servicio corre de forma aislada y es más fácil de mantener.
- **Configuración de checks:** Empezamos a añadir equipos de la red de clase y servicios web para que el programa les haga "ping" cada minuto. Si el círculo está en verde, todo va bien; si cambia a rojo, es que hay un problema de conexión.

Captura 2: Panel Uptime Kuma

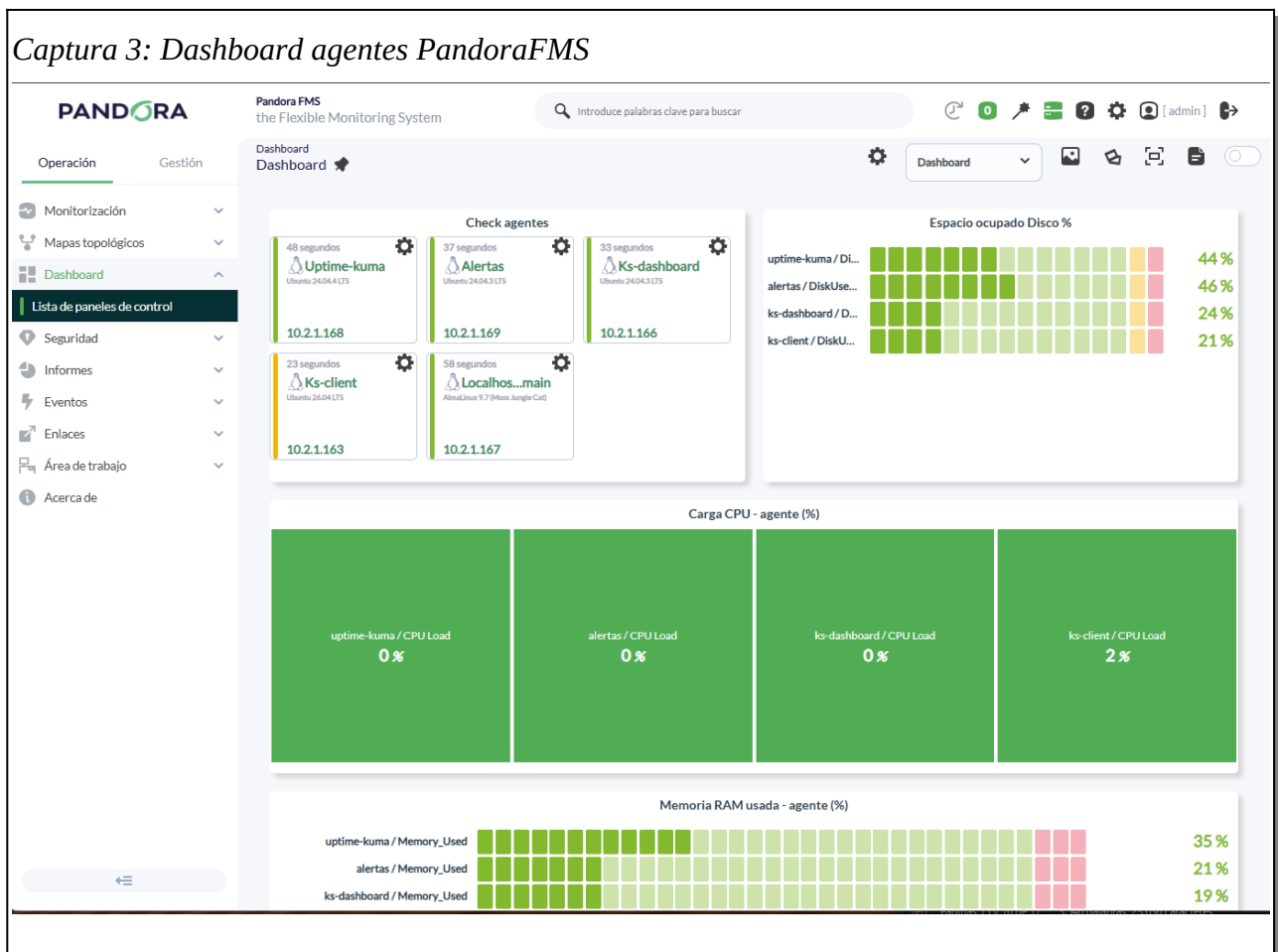


Fase 3: Monitorización avanzada con Pandora FMS

En la segunda VM instalamos **Pandora FMS**. Si Uptime Kuma nos dice si un equipo está encendido, Pandora nos dice cómo de "cansado" está ese equipo.

- **Uso de Agentes:** Instalamos pequeños programas (agentes) en las máquinas que queríamos vigilar a fondo. Estos agentes nos informan de cosas que no se ven por fuera, como si la CPU está al 100% o si el disco duro se está llenando.
- **Protocolo SNMP:** También configuramos el protocolo SNMP para poder vigilar los equipos de red del aula (como los switches), viendo en tiempo real cuánto tráfico pasa por cada puerto.

Captura 3: Dashboard agentes PandoraFMS

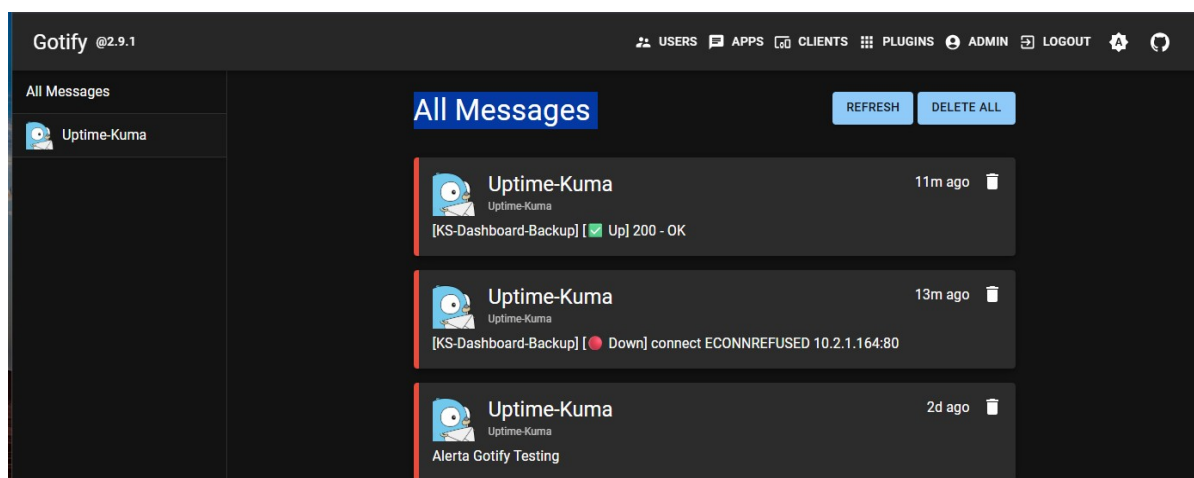


Fase 4: Integración de alertas con Gotify

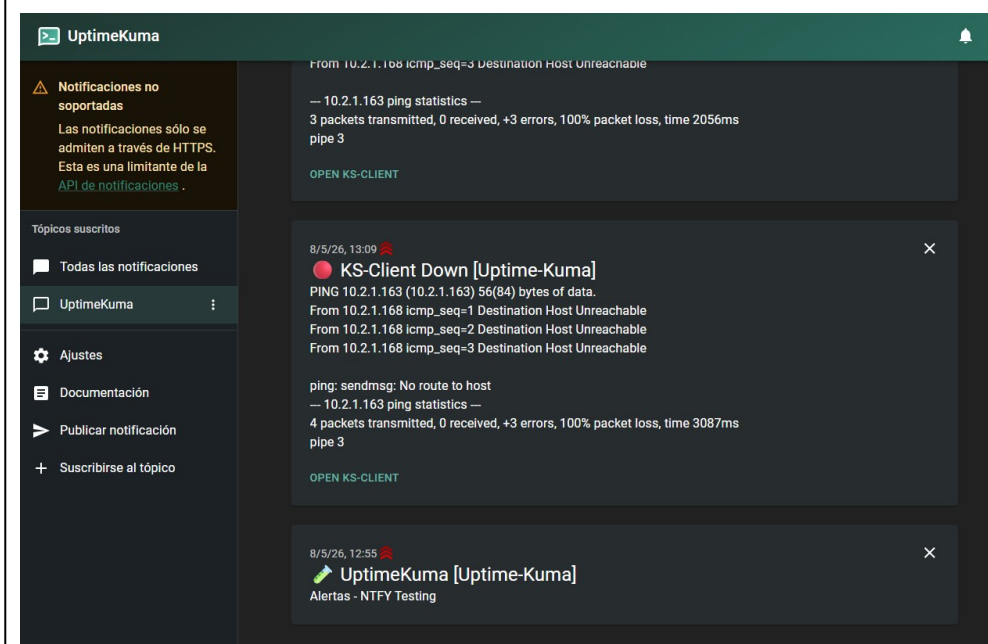
Una de las fases más importante: conectar todo para recibir los avisos. No podemos estar mirando la pantalla todo el día, así que necesitamos que el sistema nos avise al móvil.

- **Servidor de notificaciones:** Instalamos **Gotify** en la tercera VM. Es un servidor muy ligero que recibe los avisos de Pandora y Uptime Kuma y los lanza como notificaciones push.
- **Prueba de fallo:** Para verificar que todo el trabajo servía para algo, forzamos un fallo (desconectando una de las máquinas). Comprobamos con éxito que, en pocos segundos, recibíamos el mensaje de alerta en el móvil detallando qué había pasado.

Captura 4: Alertas Gotify



Captura 5: Alertas NTFY

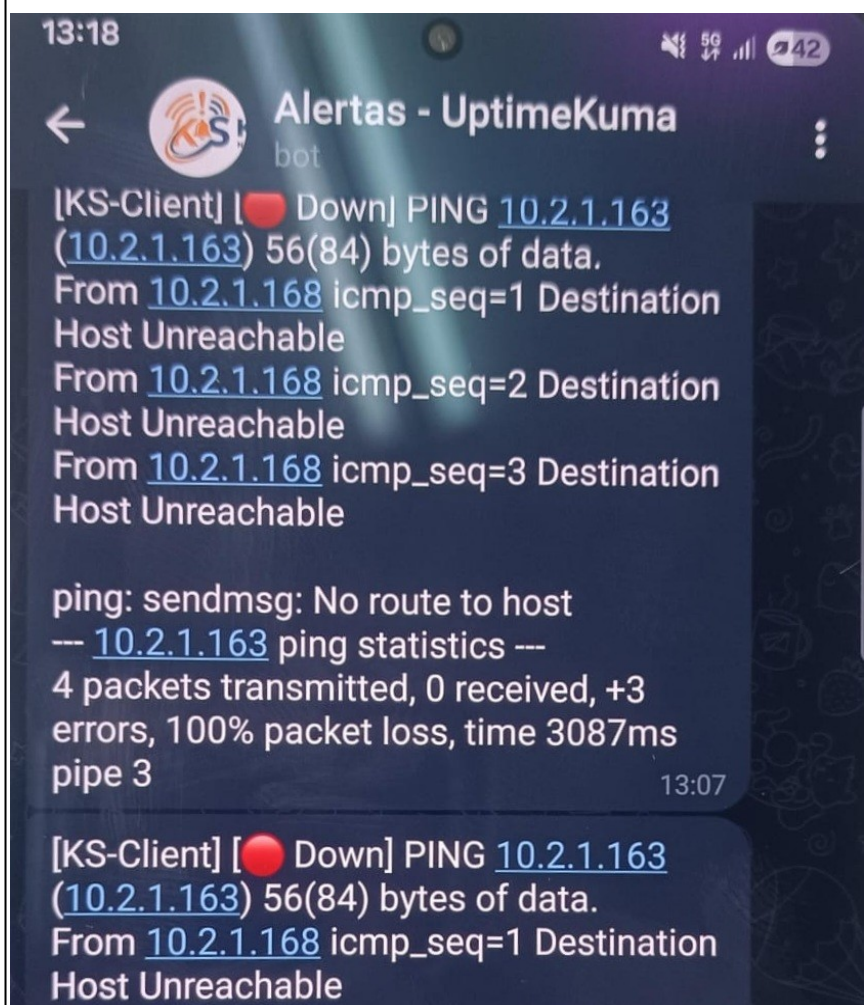


- **Configuración de bot en Telegram:** Como alternativa o complemento a Gotify, podemos integrar las alertas directamente en una app de uso diario como Telegram.

El proceso es muy sencillo: primero buscamos a **BotFather** en Telegram, iniciamos el chat y usamos el comando `/newbot` para crear nuestro propio bot, lo que nos generará un **Token** de acceso.

Después, necesitamos indicar a qué cuenta se deben enviar los mensajes. Para ello, buscamos e iniciamos el bot **User Info · Get ID · idbot**, el cual nos devolverá nuestro número de ID personal. Finalmente, introducimos ese Token y nuestro ID en los ajustes de notificaciones de Uptime Kuma o Pandora FMS. Con esto, las alertas de caídas nos llegarán como un mensaje de chat normal.

Captura 6: Alertas Telegram

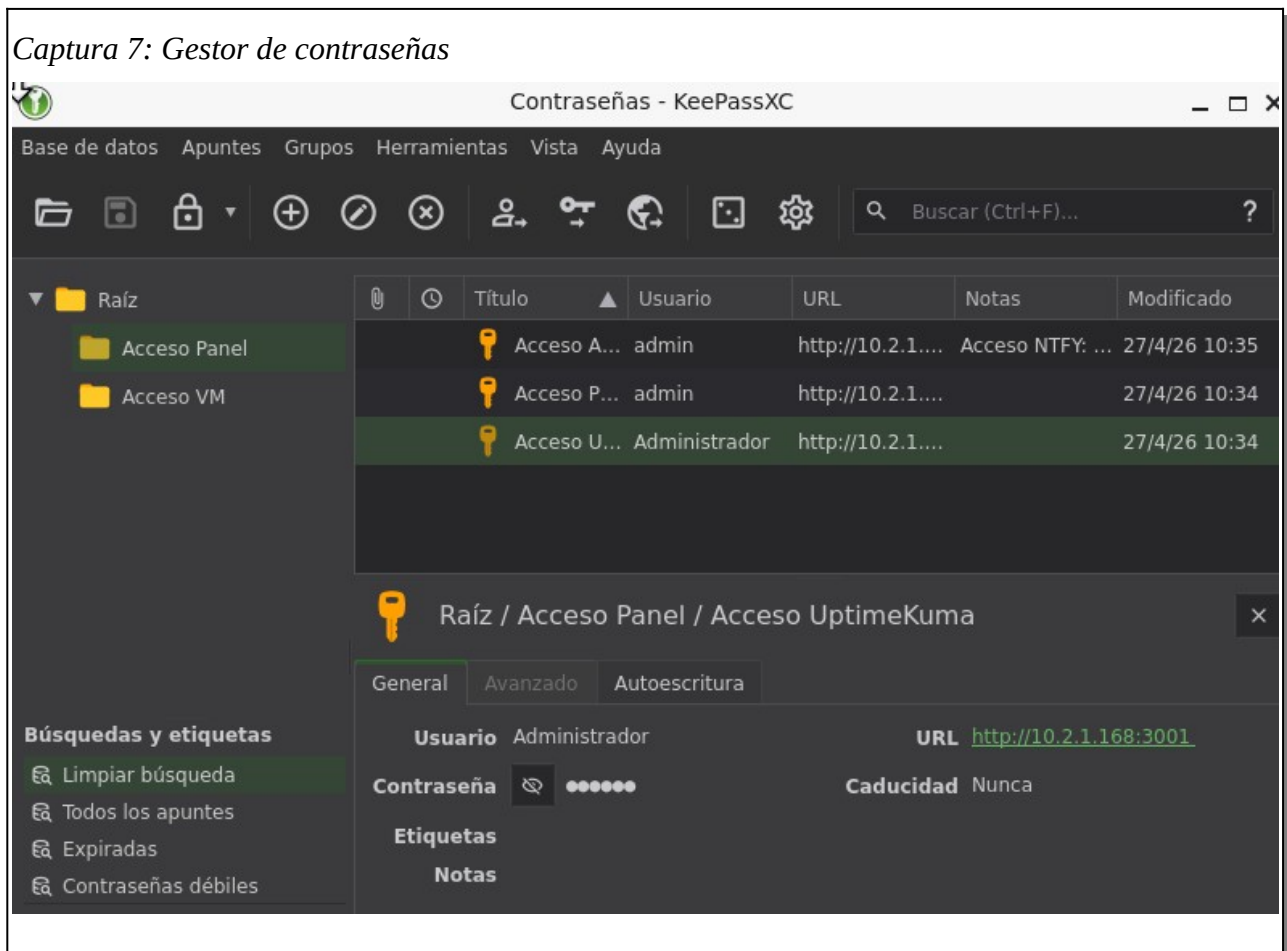


Fase 5: Gestión centralizada de credenciales con KeePass

Para cerrar el círculo de la administración segura, configuramos un entorno de gestión de accesos. Con tantas máquinas y servicios, la seguridad de las credenciales es crítica.

- **Entorno de gestión:** Desplegamos una VM específica con **Ubuntu Desktop**. En esta máquina instalamos y configuramos **KeePass**, donde almacenamos de forma cifrada todas las contraseñas de los paneles de control utilizados (Pandora FMS, Uptime Kuma, Gotify).
- **Control de acceso SSH:** Además de las interfaces web, la base de datos de KeePass incluye las claves y credenciales de acceso para la VM desde la cual realizamos las conexiones por **SSH** al resto de la infraestructura, garantizando que ninguna contraseña quede expuesta o sea olvidada.

Captura 7: Gestor de contraseñas

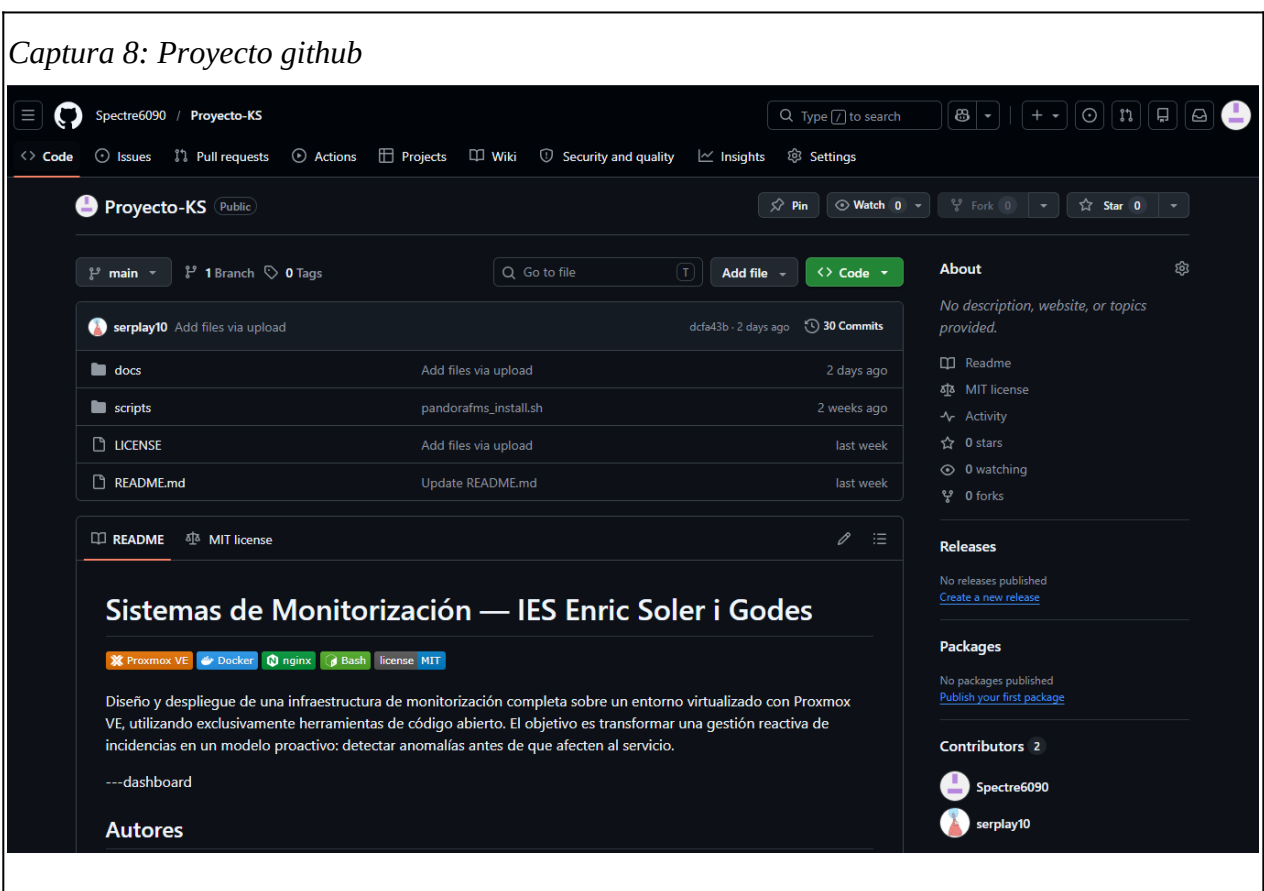


Fase 6: Documentación y repositorio en GitHub

La transparencia y la replicabilidad son pilares de este proyecto. Por ello, centralizamos todo el conocimiento generado en un repositorio público.

- **Repositorio del proyecto:** Hemos creado un proyecto en GitHub que sirve como núcleo documental. Puedes consultarlo aquí: <https://github.com/Spectre6090/Proyecto-KS>.
- **Contenido técnico:** En este repositorio almacenamos no solo los documentos de la **memoria del proyecto** detallada, sino también todos los **scripts** de automatización utilizados para montar la gran mayoría de los servicios y configuraciones, facilitando cualquier despliegue futuro o auditoría del sistema.

Captura 8: Proyecto github



6. Ampliaciones

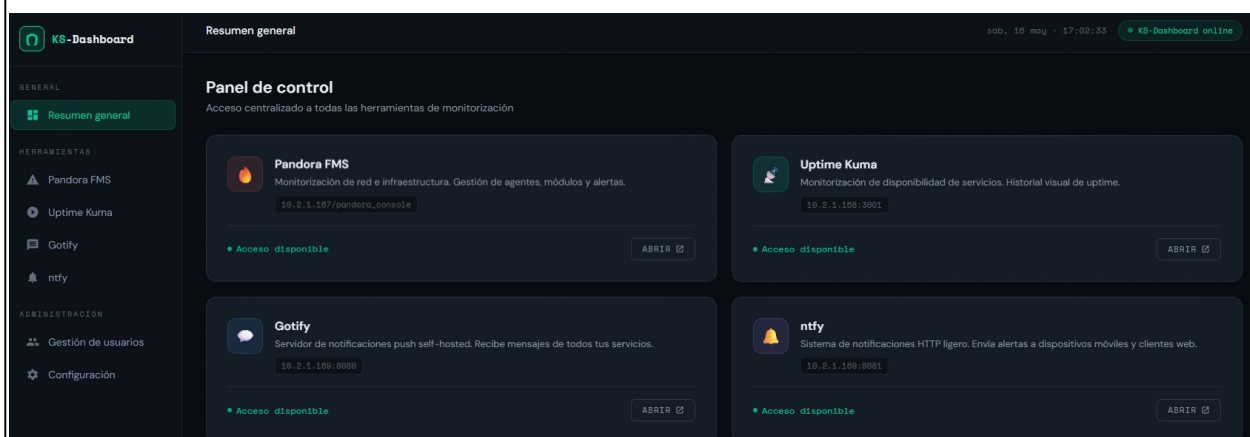
Como ampliación del proyecto, se plantea añadir un portal web centralizado llamado KS-Dashboard, desde el cual los usuarios podrán acceder a todas las herramientas de monitorización del sistema: Pandora FMS, Uptime Kuma, Gotify y ntfy. El portal estará desarrollado en HTML y CSS y será servido mediante nginx.

Para dotarlo de alta disponibilidad, se desplegarán dos servidores nginx con Keepalived, que implementa el protocolo VRRP. Uno actuará como MASTER y el otro como BACKUP, compartiendo una IP virtual (VIP) a la que accederán los usuarios. Si el MASTER cae, el BACKUP adquiere automáticamente esa IP y continúa sirviendo el portal en menos de 3 segundos, de forma completamente transparente para el usuario.

Para mantener el contenido sincronizado entre los dos nodos se utilizará lsyncd, que detecta cambios en el directorio del dashboard en tiempo real y los replica al segundo servidor mediante rsync sobre SSH.

Desde el punto de vista de la seguridad, se configurarán reglas de firewall para permitir el protocolo VRRP exclusivamente entre los dos nodos, la autenticación entre ellos estará protegida por contraseña compartida, y la sincronización entre servidores se realizará siempre mediante claves SSH, nunca con credenciales en texto plano.

Captura 9: KS-Dashboard



7. Conclusions

Este proyecto ha permitido demostrar que es posible construir un sistema de monitorización profesional, completo y funcional sin ningún coste económico, empleando únicamente herramientas de software libre y hardware donado. Partiendo de una infraestructura sin supervisión centralizada donde los problemas solo se detectaban cuando los usuarios los reportaban hemos pasado a un entorno donde cualquier caída o anomalía se detecta de forma automática y se notifica en tiempo real.

A nivel técnico, el proyecto ha supuesto un reto enriquecedor: la configuración de un entorno virtualizado con Proxmox VE, el despliegue de servicios en contenedores Docker, la integración de herramientas de monitorización como Uptime Kuma y Pandora FMS, y la implementación de un sistema de alertas push con Gotify han puesto en práctica conocimientos transversales de redes, sistemas operativos y administración de sistemas adquiridos a lo largo del ciclo.

Entre las dificultades encontradas, destaca la configuración del protocolo SNMP para la supervisión de equipos de red, la asignación correcta de IPs estáticas en el entorno virtualizado y la integración entre los distintos servicios para garantizar que las alertas llegaban correctamente al dispositivo móvil. Cada obstáculo, sin embargo, ha sido una oportunidad de aprendizaje real.

La fase de ampliación con el portal KS-Dashboard y la alta disponibilidad mediante Keepalived y lsyncd ha añadido un valor adicional al proyecto, acercándolo a una solución de nivel empresarial. La centralización de credenciales con KeePassXC y la documentación pública en GitHub reflejan un enfoque profesional y orientado a la mantenibilidad y la replicabilidad del sistema.

En conclusión, este proyecto ha sido una experiencia práctica completa que, más allá de cumplir los objetivos académicos planteados, ha sentado las bases de un sistema de monitorización real, escalable y seguro, aplicable directamente en un entorno profesional.

8. Bibliografia

Pandora FMS. (s.f.). *Pandora FMS – Software de monitorización*. Recuperado el 6 de mayo de 2026, de <https://pandorafms.com/es/>

Pandora FMS. (s.f.). *Guía de instalación de Pandora FMS*. En *Documentación oficial de Pandora FMS*. Recuperado el 6 de mayo de 2026, de https://pandorafms.com/manual/current/es/documentation/pandorafms/installation/01_installing

Lam, L. (s.f.). *Uptime Kuma* [Repositorio de software]. GitHub. Recuperado el 6 de mayo de 2026, de <https://github.com/louislam/uptime-kuma>

Uptime Kuma. (s.f.). *Uptime Kuma – A fancy self-hosted monitoring tool*. Recuperado el 6 de mayo de 2026, de <https://uptime.kuma.pet/>

UptimeKuma.org. (s.f.). *Uptime Kuma – Documentación y recursos*. Recuperado el 6 de mayo de 2026, de <https://uptimekuma.org/>

Gotify. (s.f.). *Gotify – A simple server for sending and receiving messages*. Recuperado el 6 de mayo de 2026, de <https://gotify.net/>

ntfy. (s.f.). *ntfy – Send push notifications to your phone or desktop via PUT/POST*. Recuperado el 6 de mayo de 2026, de <https://ntfy.sh/>

KeePassXC Team. (s.f.). *KeePassXC – Cross-Platform Password Manager*. Recuperado el 6 de mayo de 2026, de <https://keepassxc.org/>

9. Índice de figuras y capturas

Figura 1: Esquema general de la arquitectura del sistema de monitorización.

Captura 1: Vista de las máquinas virtuales (VMs) desplegadas en el entorno Proxmox.

Captura 2: Panel de control de Uptime Kuma mostrando el estado de disponibilidad de los servicios.

Captura 3: Dashboard de agentes en Pandora FMS con métricas de rendimiento.

Captura 4: Interfaz de Gotify con el historial de alertas y notificaciones push recibidas.

Captura 5: Interfaz de NTFY con el historial de alertas y notificaciones push recibidas..

Captura 6: Bot de Telegram con el historial de alertas y notificaciones push recibidas

Captura 7: Gestor de contraseñas KeePassXC con las credenciales cifradas del proyecto.

Captura 8: Repositorio oficial del proyecto en GitHub con los scripts y documentación.

Captura 9: Accesos directos desde el portal centralizado KS-Dashboard, mostrando la interfaz web que unifica la entrada a todas las herramientas de monitorización (Pandora FMS, Uptime Kuma, Gotify y ntfy).